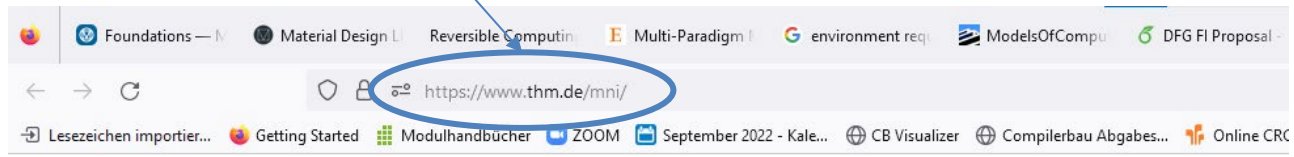
1.	Organisatorisches	4
2.	Einleitung	21
3.	**HTML**	**46**
4.	CSS	70
5.	Frameworks	89
6.	HTTP	106
7.	Internet	126
8.	TypeScript	178
9.	HTML+CSS+Typescript	224
10.	Web Services	235
11.	REST-Services	258

Woher kommt der Inhalt auf meinen Browser?

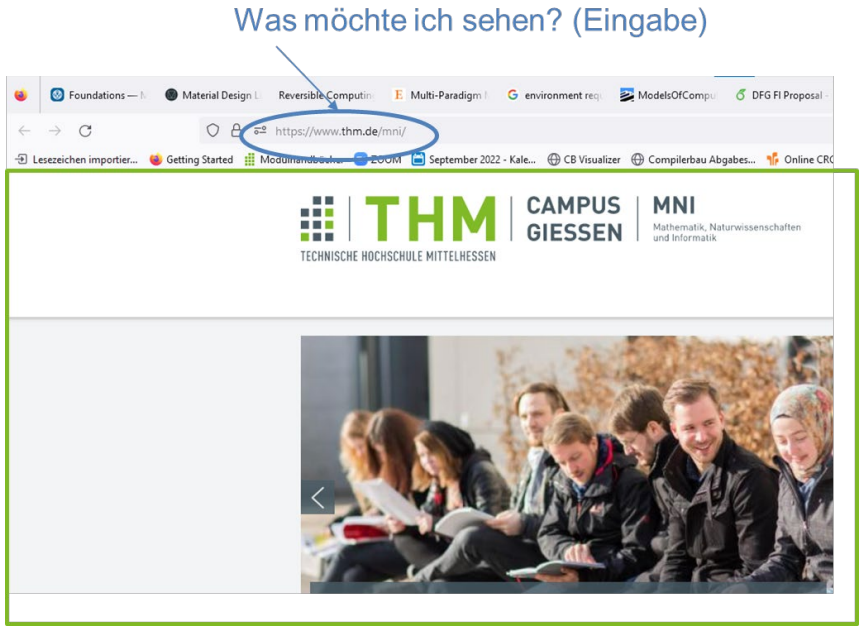
Was möchte ich sehen? (Eingabe)





Inhalt (Ausgabe)

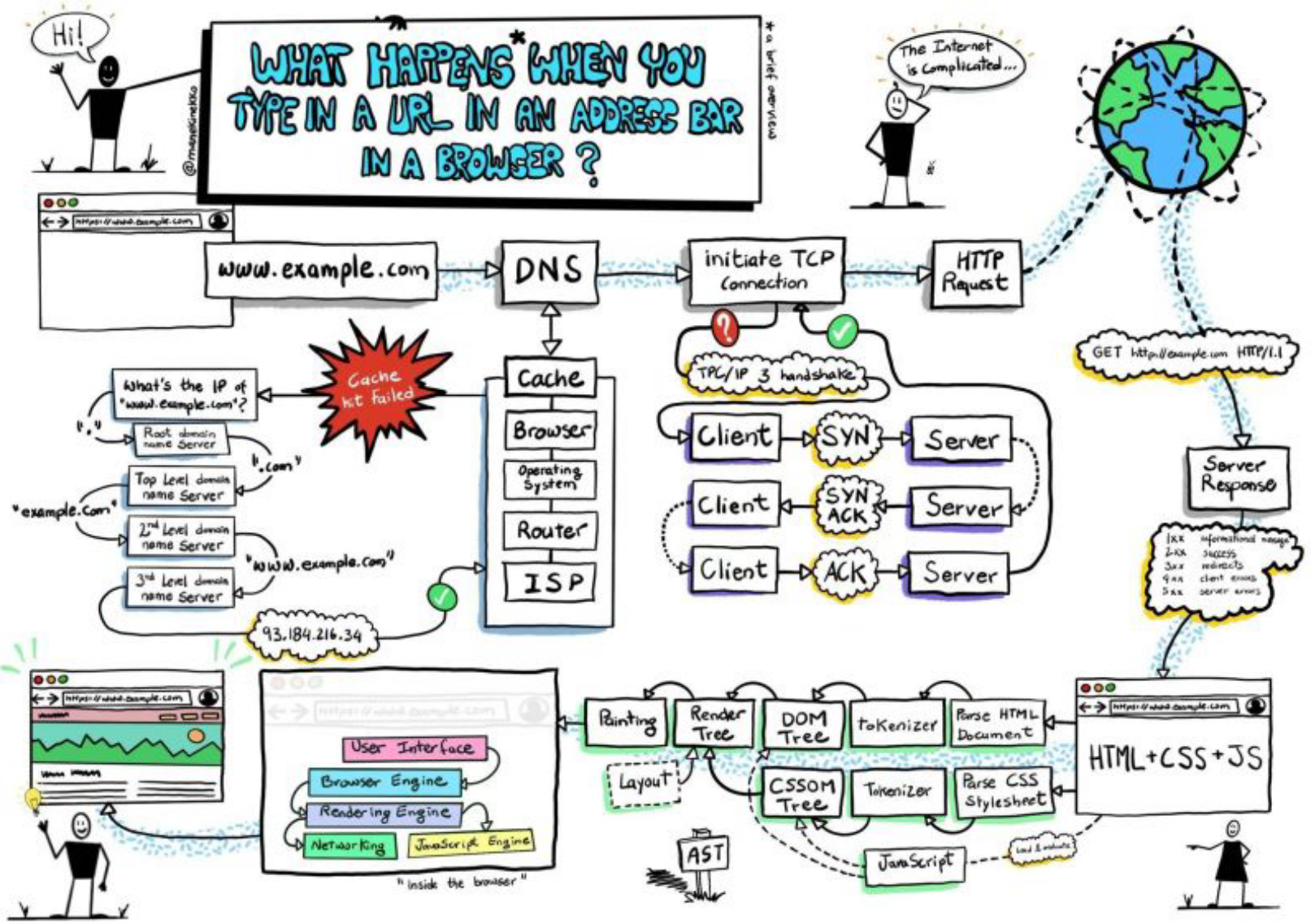
Woher kommt der Inhalt auf meinen Browser?



http:// -> Name eines **Protokolls**
www.thm.de -> Name eines Rechners („**Host**“)
/mni -> Name einer Datei („**Ressource**) auf diesem Rechner

Host und Ressource zusammen werden als **URL** („Uniform Resource Locator“) bezeichnet.

Die URL bestimmt eindeutig, was der Benutzer sehen möchte.
Aber woher kommt der Inhalt?


Hi!
WHAT HAPPENS* WHEN YOU TYPE IN A URL IN AN ADDRESS BAR IN A BROWSER ?
* a brief overview
@mattgreenrocks
The Internet is Complicated...
www.example.com
DNS
initiate TCP connection
HTTP Request
TPC/IP 3 handshake
Client
SYN
Server
Client
SYN ACK
Server
Client
ACK
Server
GET http://example.com HTTP/1.1
Server Response
Cache
Browser
Operating System
Router
ISP
Cache hit Failed
What's the IP of "www.example.com"?
Root domain name Server
".com"
Top Level domain name Server
"example.com"
2nd Level domain name Server
"www.example.com"
3rd Level domain name Server
93.184.216.34
1XX informational message
2XX success
3XX redirects
4XX client errors
5XX server errors
HTML+CSS+JS
Painting
Render Tree
DOM Tree
tokenizer
Parse HTML Document
Layout
CSSOM Tree
Tokenizer
Parse CSS Stylesheet
JavaScript
AST
User Interface
Browser Engine
Rendering Engine
Networking
JavaScript Engine
"inside the browser"

index.html

```
<!doctype html>
<html>
    <head>
       <style> … </style>
       <script> … </script>
    </head>
    <body>
          VIEL TEXT HIER!
     </body>
</html>
```

Darstellung im Browser (Rendering)



HTML zur strukturierten Beschreibung der Webseiteninhalte (des **Contents**)

CSS (**Stylesheets**) zur Beschreibung der Darstellung

JavaScript/TypeScript als Programmiersprache, um Webseiten mit Funktionalität auszustatten

HTML

HTML

Hypertext

Nichtlineare, netzartige Texte.

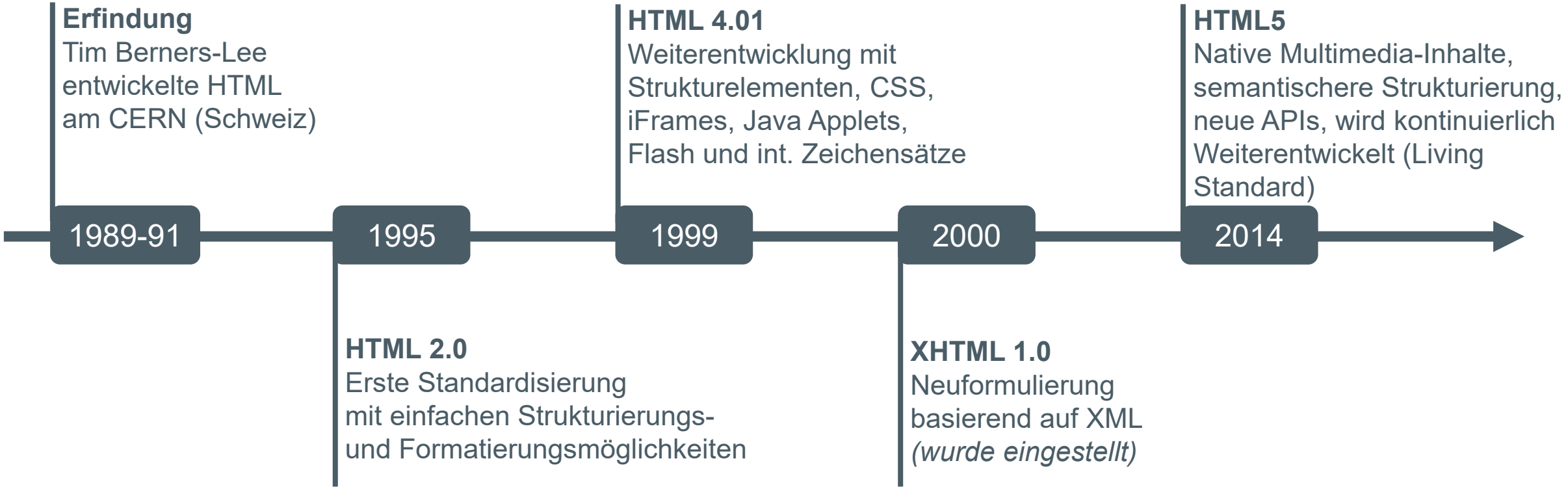
Markup-Language

„Auszeichnungssprache“: Strukturen und Eigenschaften des Textes werden durch „Tags“ ausgezeichnet.

Moby Dick
Autor: Herman Melville
Jahr: 1851

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <title>Moby Dick</title>
</head>
<body>
<em>Autor: </em>
   <img src="https://melville.gif">
   Herman Melville
<br>
<em>Jahr: </em>1851
</body>
</html>
```

Kurze Historie zu HTML



Anleitung für die Folien



Ausprobieren!

HTML, CSS und TypeScript lernen Sie nicht durch Auswendiglernen!

NICHT!

Probieren Sie nach der Vorlesung alle Beispiele selbst auf Ihrem Rechner aus!

Verändern Sie die Beispiele und beobachten Sie was passiert.

HTML Grundgerüst

Jeder HTML-Code sollte in einer Datei mit Endung `.html` stehen,

```
<!DOCTYPE html>
<html lang="de">
  <head>
    <title>Page Title</title>
  </head>
  <body>
    Es war einmal …
  </body>
</html>
```

`DOCTYPE` bestimmt die HTML Version (hier: Living Standard)
`html lang` bestimmt die Sprache (hier: deutsch)

Header enthält Metadaten, Styles und Scripts
Metadaten = Informationen über Daten, die die Daten nicht selbst enthalten (z.B. Browser-Einstellungen).

Body: Inhalt
Text, Verweise auf Bilder, Videos, …

HTML Tags

Tags steuern die Darstellung des Inhaltes und können als Paar stehen („geschlossen“) oder einzeln („offen“)

- **Geschlossene** Tags: `<table></table>, <head></head>`
- **Offene** Tags: `<img>, <br>,`

Tags können **ein** oder **mehrere Attribute** enthalten

- Manche Attribute sind in jedem Tag verfügbar (universell)
- Manche Attribute sind nur für bestimmte Element zugelassen (speziell)

```
<Tagname attribut1="wert1" attribut2="wert2" >
```

Attribute dienen

- zur genaueren Kennzeichnung von Tags
- zur graphischen Gestaltung
  (Diese sollten, wenn möglich, durch CSS – Cascading Stylesheets – ersetzt werden)

Komplette Attribut-Referenz: https://www.w3schools.com/tags/default.asp

Attribute

- Ein **Attribut** ist ein **Schlüssel-Wert Paar**:
  `<name attribut1="wert1" attribut2="wert2">`
  - Beispiele:
    `<span class="author"></span>`
    `<meta charset="utf-8">`

- Einige Attributwerte müssen **eindeutig** sein
  - z.B. `<span id="time">`
    "`time`" darf nicht mehr woanders als Wert für `id` benutzt werden

- Attribute können auch anders angegeben werden:
  - Ohne Angabe des Schlüssels: `<Tagname wert>`
  - Wert ohne Anführungszeichen: `<Tagname attribut=wert>`
  - Wert mit einfachen Anführungszeichen: `<Tagname attribut='wert'>`

Häufig verwendete Attribute

Attribut	Beschreibung
`alt`	Alternativtext für Bilder, wenn Bild nicht vorhanden
`disabled`	Deaktiviert Input-Elemente
`href`	Definiert die Web-Adresse eines Links
`id`	Eindeutiger Bezeichner für ein Element
`src`	Definiert den Pfad (URL) zu einem Bild
`style`	Style-Angaben (CSS) für ein Element
`title`	Weitere Informationen zu einem Element (wird als Tooltip angezeigt)

- Komplette Attribut-Referenz: http://www.w3schools.com/tags/ref_attributes.asp

HTML Elemente

Ein HTML Element

- ist eine Struktur innerhalb eines HTML Dokumentes
- Besteht aus
  - Tags
  - Inhalt
  - Attributen

Beispiel:

```
<p id=abschnitt1 class=hervorgehobenerText>
   Hier steht was Wichtiges!
</p>
```

Block- und Inline-Elemente

- **Blöcke** werden **alleine** in eine **Zeile** mit **voller Breite** positioniert
  - dürfen in der Regel verschachtelt werden
  - Beispiele: `div, h1 - h6, p, …`
- **Inline**: positionieren sich nicht in eine extra Zeile und nehmen nur so viel Platz ein, wie sie brauchen
  - sind Bestandteil des Textes
  - Beispiele: `span, a, img, …`

```
<!DOCTYPE html>
<html lang="de" >
<head>
  <title>Rapunzel</title>
  <meta charset="UTF-8">
</head>
<body>
  <h1>Rapunzel</h1>                                  Block
  <span>Jacob Grimm (1785-1863)</span>               Inline
  <span>Wilhelm Grimm (1786-1859)</span>             Inline
  <p>Es war einmal ein ...</p>                       Block
  <p>Die Leute hatten ...</p>                        Block
</body>
</html>
```

Rapunzel

Jacob Grimm (1785-1863) Wilhelm Grimm (1786-1859)
Es war einmal ein ...

Die Leute hatten ...

Block- und Inline-Elemente

Block	
Element	**Beschreibung**
`div`	Gruppierung
`p`	Absatz
`h1 - h6`	Überschriften
`address`	Kontaktinformation
`article`	Inhalt eines Artikels
`dl, li, ul, …`	Listen
`form`	Formular
`table`	Tabelle
`hr`	Trennlinie

Inline	
Element	**Beschreibung**
`span`	Gruppierung
`b`	Spez. Bedeutung
`em`	Wichtig
`mark`	Hervorgehoben
`a`	Anker
`br`	Zeilenumbruch
`img`	Bild
`code`	Quellcode
`button`	Knopf

Komplette Referenz : https://www.w3schools.com/html/html_blocks.asp

Hyperlinks

- Der Anker („anchor“) definiert einen Hyperlink
  - zu externer Seite
  - innerhalb der Seite
  - zu einem Programm
- `href`: definiert das Ziel
  - als URI:
    - `www.thm.de/bau/` (absolut) oder `/bau` (relativ) — öffnet die Seite des Fachbereichs BAU
    - `mailto:max.mustermann@gibts.net` — öffnet E-Mail Client
  - als Name:
    - `#` — spring zum Anfang
    - `#ziel` — spring zum Element mit der `id` “`ziel`”
- `<a>Name</a>` entspricht `<a href="#">Name</a>`
- `<a href="mailto:luke.skywalker@yedi.org">E-Mail-Kontakt</a>`

Listen

Ungeordnete Liste

```
<ul>
  <li>Meyer</li>
  <li>Priefer</li>
  <li>Süß</li>
</ul>
```

- Meyer
- Priefer
- Süß

Geordnete Liste

```
<ol>
  <li>Hören</li>
  <li>Denken</li>
  <li>Sprechen</li>
</ol>
```

1. Hören
2. Denken
3. Sprechen

Definitions-liste

```
<dl>
  <dt>P1</dt>   <dd>Programmieren 1<dd>
  <dt>PI1</dt>  <dd>Praktische Informatik 1</dd>
  <dt>Te1</dt>  <dd>Technische Informatik 1</dd>
</dl>
```

P1
Programmieren 1
PI1
Praktische Informatik 1
Te1
Technische Informatik 1

Verschachtelte Listen

```
<ul>
  <li>Tee</li>
  <li>Kuchen
    <ul>
      <li>Apfelkuchen</li>
      <li>Kirschkuchen</li>
    </ul>
  </li>
  <li>Kaffee</li>
</ul>
```

- Tee
- Kuchen
  - Apfelkuchen
  - Kirschkuchen
- Kaffee

Tabellen

```
<table>
    <caption>Stundenplan</caption>
    <tr>
        <th>Block</th>
        <th>Mo</th>
        <th>Di</th>
        <th>Mi</th>
        <th>Do</th>
        <th>Fr</th>
    </tr>
    <tr>
        <td>1</td>
        <td>P1</td>
        <td>PI1</td>
        <td>Ma1</td>
        <td></td>
        <td>TE1</td>
    </tr>
    <tr>
        <td>2</td>
        <td>SWT</td>
        <td></td>
        <td>BS</td>
        <td>KSP</td>
        <td></td>
    </tr>
</table>
```

`th`: Table Header
`tr`: Table Row (Zeile)
`td`: table data

Stundenplan

Block	Mo	Di	Mi	Do	Fr
1	P1	PI1	Ma1		TE1
2	SWT		BS	KSP	

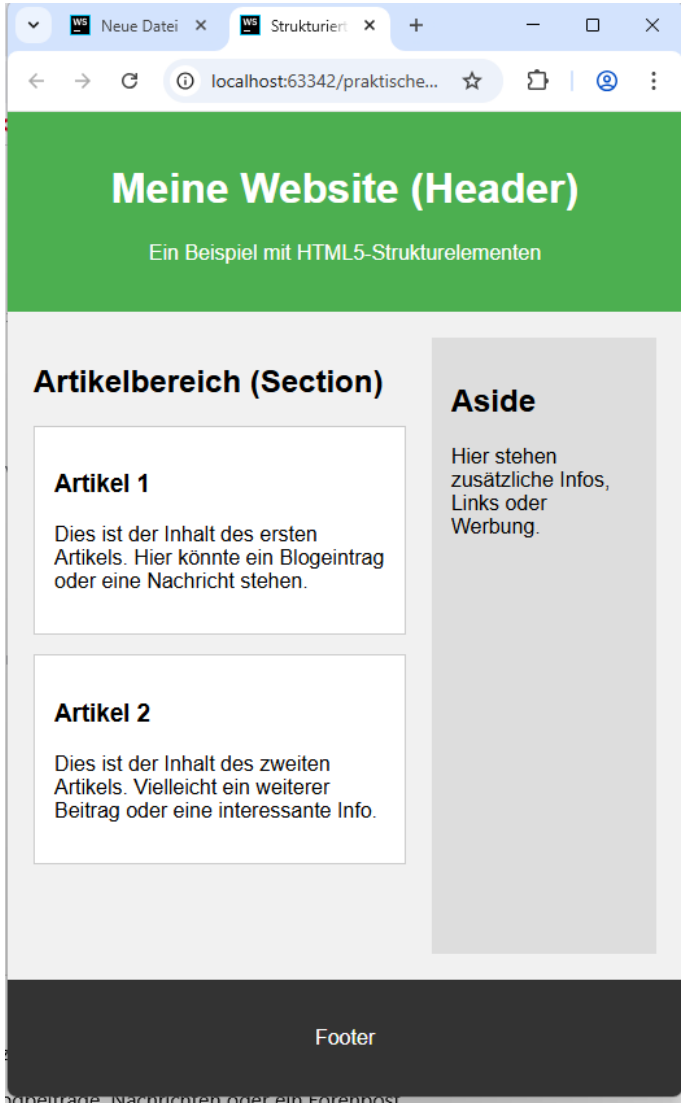
Semantische Elemente (ab HTML 5)

Semantische Elemente haben eine klar-definierte Bedeutung (Semantik) und können die Entwicklung vereinfachen und die Lesbarkeit verbessern.

Folgende Elemente stehen zur Verfügung:

- <article>
- <aside>
- <details>
- <figcaption>
- <figure>
- <footer>
- <header>
- <main>
- <mark>
- <nav>
- <section>
- <summary>
- <time>





http://www.w3schools.com/html/html5_semantic_elements.asp

Formulare

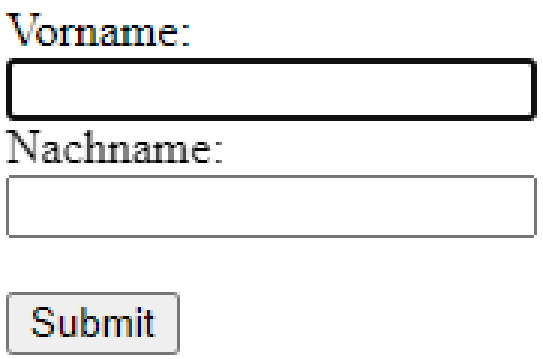
Eingabefeld

Eingabefeld

```
<div>
    Vorname:<br>
    <input type="text" name="vorname"><br>
    Nachname:<br>
    <input type="text" name="nachname"><br><br>

    <button type="button"
    onclick="alert('HelloWorld!')">Submit</button>
</div>
```

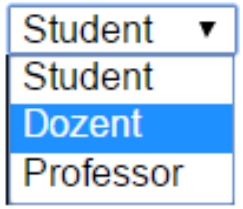
Button zum Ausführen von JavaScript



Weitere Formularelemente

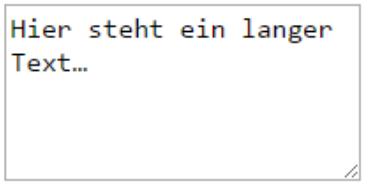
- Auswahlliste (**Drop-Down**)

```
<select name="status">
  <option value="student">Student</option>
  <option value="dozent">Dozent</option>
  <option value="prof">Professor</option>
</select>
```



- Textfeld (**Textarea**)

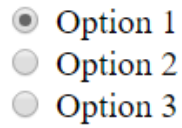
```
<textarea name="nachricht" rows="5" cols="20" >
  Hier steht ein langer Text…
</textarea>
```

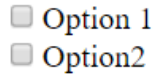


- Eingabe-Typen (**Input**)

```
<input type="text" name="vorname">
<input type="password" name="pw">
<input type="radio" name="r1">
<input type="checkbox" name="cb1">
```







Neue Typen:

```
<input type="color" name="lieblingsfarbe">, <input type="date" name="geburtstag">
```

→ Typen, welche vom Browser nicht unterstützt werden, werden als `<input type="text">` interpretiert

Document Object Model (DOM)

Der Browser muss ein HTML Dokument **parsen** und anschließend darstellen

Der **Parser** liest die Zeichen des HTML Dokumentes und erstellt daraus einen **Baum**

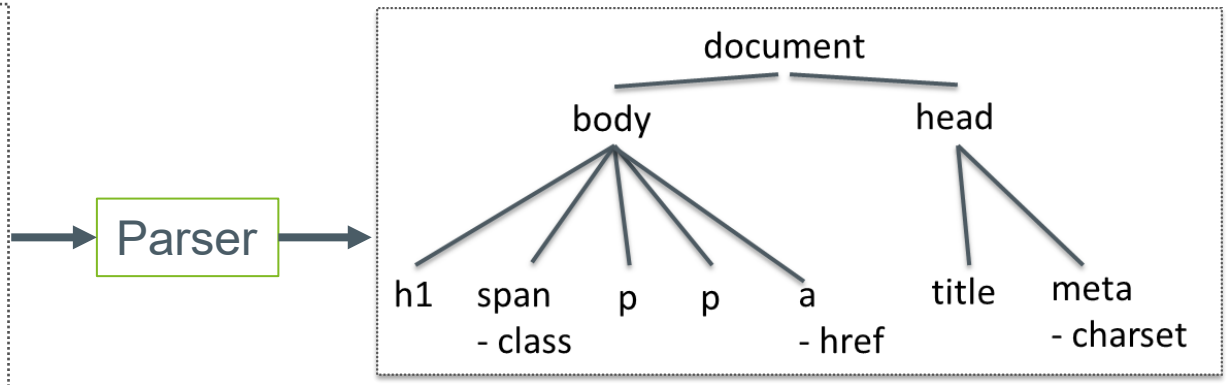
→ Diesen Baum nennt man: *Document Object Model (DOM)*

```
<!DOCTYPE html>
<html lang="de">
  <head>
    <title>Rapunzel</title>
    <meta charset="UTF-8">
  </head>
  <body>
    <h1>Rapunzel</h1>
    <span class="author">...</span>

    <p>...</p>

    <p>...</p>

    <a href="#">...</a>
  </body>
</html>
```



Auf diesen Baum kann man mit JavaScript zugreifen und ihn manipulieren